AIPL Runtime Feature Matrix

7 implementations compared

Yasushi Kodama

2026-05-13

Overview

AIPL (Actor-based Intelligent Parallel Language, formerly ABCL/c+) has grown into a **seven-runtime project**. This document provides a side-by-side comparison of every language and infrastructure feature across all implementations, grounded in source surveys.

Legend. ✓ = full support, △ = partial, × = not supported, – = not applicable.

The seven runtimes.

Short name	Source
Python (annotated)	`src/python-aipl/`
Python (inferred)	`src/python-aipl-inferred/`
OCaml	`src/*.ml`, `_build/.../repl_thread.exe`
JS-OCaml (server)	`src/app.js`, `console_server.js`, `ide.js` + OCaml `web_gateway`
JS-Browser (serverless)	`src/browser-abcl/`
JS-Node (server)	`src/node-aipl-server/` (Node HTTP server wrapping browser-abcl)
C (abcl2c)	`src/abcl2c.ml` codegen + `src/abcl_gui_runtime.c`

Column abbreviations in matrices below. **Py-A** = Python annotated, **Py-I** = Python inferred, **OCaml**, **JS-O** = JS via OCaml backend, **JS-B** = JS in browser, **JS-N** = JS in Node server, **C** = abcl2c codegen.

Inheritance relationships.

- **JS-OCaml** is a thin HTTP client over the OCaml backend, so its feature set inherits OCaml's (plus the new `/api/typecheck` endpoint).
- **JS-Node** wraps the same browser-abcl parser / type-checker / interpreter inside a Node.js HTTP server. Runtime feature set therefore mirrors JS-Browser.
- **Python (inferred)** imports parser, interpreter, AI, remote, and dashboard from `../python-aipl/`. All *runtime* features match Python (annotated); only the *static type-checker* layer differs (full Hindley-Milner instead of Phase 11+ annotation-driven).

Sample programs

Number of `.abcl` sample programs reachable by each runtime (core directory + AI integration + remote-actor demos):

	Py-A	**Py-I**	**OCaml**	**JS-O**	**JS-B**	**JS-N**	**C**
core samples	33	33	57	57	5	5	57
AI integration samples	11	11	8	8	0	0	8
remote-actor samples	8	8	1	1	0	0	1
total	**52**	**52**	**66**	**66**	**5**	**5**	**66**

Cross-cutting / not tied to a specific runtime:

- `aipl-self-host/`: 48 AIPL-in-AIPL self-host programs
- `docker/cross/samples/`: 4 cross-language interop samples

Note that the C runtime processes the same `.abcl` files as OCaml via `abcl2c`; of the 66 reachable samples, 48 currently pass the `abcl2c` smoke test (the remaining 9 in `abclc/` have pre-existing type ambiguities surfaced by our hard-fail policy).

What each sample exercises

Samples are grouped by the language or infrastructure feature they demonstrate. "Where" points to the source directory; "Target" lists the runtime(s) and codegen targets that actually execute the sample.

Target abbreviations: **Py** = python-aipl / python-aipl-inferred; **OCaml** = OCaml REPL (`abclc`); **C** = `abcl2c` → C+pthread; **SDL2** = `abcl2c` → C+SDL2 GUI binary; **Xinu** = `abcl2c --xinu` → Xinu-flavoured C; **Py-gen** = `abcl2c --python` → stand-alone Python; **Pony** = `abcl2c --pony` → Pony actor source; **Erlang** = `abcl2c --erlang` → Erlang module; **Go** = `abcl2c --go` → Go source; **JS-B** = browser-abcl; **JS-N** = node-aipl-server.

Feature category	**Sample(s)**	**Where**	**What it checks**	**Target**
Basic actor model	`Hello`, `Counter`, `PingPong`	py-aipl + abclc	actor creation, `send`, field state, `self`-send	Py, OCaml, C, JS-B, JS-N
`now`/`future`/`await`	`NowFuture`, `CooperativeNowFuture`, `NowFutureDemo`	py-aipl + samples-ai + abclc/ai-samples	three message-passing forms in one program	Py, OCaml, C, JS-B, JS-N
Bounded mailboxes / select	`BoundedBuffer`, `bounded_buffer{,_visual}`	py-aipl + abclc + browser-abcl	producer/consumer, selective receive, visualisation	Py, OCaml, C, JS-B, JS-N
Dining philosophers	`Philosophers`, `philosophers{-1,-2}`, `Philosophers5{,_debug,_trace,Gui,Py,Xinu}`	py-aipl + abclc + browser-abcl	fork as actor, deadlock-free serialisation, SDL/Python/Xinu variants	Py, OCaml, C, SDL2, Py-gen, Xinu, JS-B, JS-N
`become` (class swap)	`become`, `bbecome`	abclc	runtime actor-class replacement	Py, OCaml
`select` / selective recv	`Channels`, `Channels2`	py-aipl/samples	typed channels, worker pool with request/result	Py
Method injection	`MethodPatch`	py-aipl/samples	`add_method`/`remove_method` runtime patching	Py

Feature category	Sample(s)	Where	What it checks	Target
Dynamic compile	`Dynamic`, `DynamicWorkerPool`	py-aipl/samples	`compile()` builtin → runtime class generation	Py
Top-level functions	`Functions`, `Signatures`	py-aipl/samples	user functions, overload signatures, typeof	Py
Records	`Records`	py-aipl/samples	`{a:int, b:string}` literal, field access, structural typeof	Py
Tuples	`Tuples`	py-aipl/samples	positional, immutable, mixed slot types, nesting	Py
Arrays	`Arrays`, `MultiDimArrays`	py-aipl/samples	static-sized, multi-dim, dynamic sizes	Py
Gradual type checker	`Typecheck`, `Typecheck11{b,c,de}`	py-aipl/samples	Phase 11 → 11e: literals, call-site validation, unions/generics, narrowing, length-tagged arrays	Py
Phase 11 typed counter	`Phase11_TypedCounter`	abclc	typed annotations / generics / typeof narrowing	OCaml
Phase 12 effects	`Effects`, `Phase12_EffectsLog`	py-aipl + abclc	capability-based `!{fs,ai,net,mut}` system	Py (static), OCaml (sample)
Phase 13 channels	`Phase13_Channels`	abclc	CSP channels (design doc + runtime in Py)	Py (runtime), OCaml (sample)
Phase 14 linear types	`Linear`, `Linear2`, `Phase14_Linear`	py-aipl + abclc	use-after-move; DB transaction with linear handle	Py (static), OCaml (sample)
Phase 15 owned fields	`Owned`, `Owned_violations`, `Phase15_Owned`	py-aipl + abclc	`pub` field visibility, intentional violations	Py (static), OCaml (sample)
Phase 16 transient cast	`Transient`, `Transient_violation`	py-aipl/samples	runtime type cast at any-boundary	Py
Phase 17 structured conc.	`Phase17_StructuredConc`	py-aipl/samples	`scope { future ... }` auto-joining	Py
Session types / protocol traces	`SessionTyped`	py-aipl/samples-ai	runtime-checked typed session protocols (arg + reply types); order-only protocol traces; AIOS service registry	Py
AI integration (mock + real)	`AIActor`, `AIChain`, `AIChainReal`, `AIHello`, `MultiProvider`	py-aipl + samples-ai + abclc/ai-samples	LLM-backed actors; multi-provider; chained pipeline	Py, OCaml
AI governance	`Budgeted`	py-aipl/samples-ai + abclc/ai-samples	token budget, concurrency cap, fallback chain	Py, OCaml
AI cooperative pattern	`CooperativeNowFuture{,-jp,-jp-remote}`, `CooperativeSolve{,-jp,Remote,Remote-jp}`, `Reviewer`, `Fanout`, `PriorityFanout`	py-aipl/samples-ai, abclc/ai-samples	Planner→Solver→Reviewer; fan-out aggregator; priority routing	Py, OCaml

Feature category	Sample(s)	Where	What it checks	Target
Remote actors	`client`, `server`, `coordinator`, `solver`, `verifier`, `reviewer_node*`, `RemoteCalcClient/Server`	py-aipl/samples-remote + abclc/samples-remote + abclc/ai-samples	HTTP cross-machine sends; HMAC-signed coordination	Py, OCaml
Web / dashboard	`web_calc`, `SiteGen`	abclc + py-aipl/samples	embedded HTTP gateway; static-site generator	Py (SiteGen), OCaml (web_calc)
GUI / SDL2	`Rotate{One,Three,Four}Lines{,Gui}`, `MultiLineSpin`, `Philosophers5Gui`, `BoundedBufferGui`, `DisasterReturnGui`, `LineDrawer`, `window`	abclc	SDL2-backed GUI codegen via abcl2c	SDL2 (via abcl2c)
Python codegen target	`BoundedBufferPy`, `Philosophers5Py`, `Rotate4LinesPy`	abclc	`abcl2c --python` emits stand-alone Python	Py-gen
Xinu (embedded OS) target	`BoundedBufferXinu`, `Philosophers5Xinu`, `Rotate4LinesXinu`	abclc	`abcl2c --xinu` emits Xinu-flavoured C	Xinu
Pony codegen target	`Hello`, `counter` (verified); `PingPong` (xfail)	abclc	`abcl2c --pony` emits Pony source; `class` → `actor`, methods → `be`; two-step `_aipl_init` decouples construction from init body	Pony
Erlang codegen target	`Hello`, `counter` (verified); `PingPong` (xfail)	abclc	`abcl2c --erlang` emits an `.erl` module; `class` → `spawn` + `receive` loop; fields → loop args with versioned variables on assign	Erlang
Go codegen target	`Hello`, `counter` (verified); `PingPong` (xfail)	abclc	`abcl2c --go` emits a single `main.go`; `class` → struct + goroutine; methods → typed message structs + helper methods + type-switch dispatch	Go
Drone / simulation	`drone_simulator`	browser-abcl	obstacle-aware drone swarm with comm + view range	JS-B, JS-N
Trace / minimal repros	`H`, `P`, `T*`, `LD*`, `MS`, `AA`, `PP`, `PH`, `line*`, `Philosophers5_{debug,trace}`	abclc	reduced repro cases used during runtime / TLA+ / Spin model-checking	OCaml

Core language

Feature	Py-A	Py-I	OCaml	JS-O	JS-B	JS-N	C
method injection (`add_method`/`remove_method`)	✓	✓	×	×	×	×	×
`now` synchronous send	✓	✓	✓	✓	✓	✓	✓
`future` async send	✓	✓	✓	✓	✓	✓	✓
`await` future block	✓	✓	✓	✓	✓	✓	✓
`send` fire-and-forget	✓	✓	✓	✓	✓	✓	✓
`become` actor class swap	✓	✓	✓	✓	×	×	×
`select` selective receive	✓	✓	✓	✓	✓	✓	×
top-level functions	✓	✓	△	△	×	×	✓
signatures / overloads	×	×	✓	✓	×	×	×
dynamic compile (`compile()`)	✓	✓	×	×	×	×	×
worker pool / `DynamicWorkerPool`	✓	✓	×	×	△	△	×

Type system

Feature	Py-A	Py-I	OCaml	JS-O	JS-B	JS-N	C
type inference	△trace	✓HM	✓HM	✓HM	✓flow	✓flow	✓HM+spec
type annotations	✓	△ignored	×	×	×	×	×
records `{a:int, b:str}`	✓	✓	△type only	×	×	×	△type only
tuples `(1, "a")`	✓	✓	×	×	×	×	×
arrays (typed, multi-dim)	✓	✓	✓	✓	△	△	△
generics on functions	×	✓Forall	△Forall	△	×	×	×

Phase 11+ advanced features

Python (inferred) runs HM only, so the annotation-driven Phase 11+ static checks that Python (annotated) performs are not re-implemented. Programs still execute at runtime via the shared interpreter, but the static guarantees from Phase 12 / 14 / 15 / 16 are lost. Phase 13 (channels), Phase 17 (`scope`), and the session-type / protocol-trace / AIOS-registry families are runtime features, so they remain available in Py-I.

Session types here are *runtime-checked* and explicitly separate from HM inference: they observe values flowing over the wire at runtime and validate them against a declared protocol spec (`"actor.method(arg_t1, ...) ! ret_type -> ..."`). Violations are recorded into `session_events()` and counted; the program does not abort.

Feature	**Py-A**	**Py-I**	**OCaml**	**JS-O**	**JS-B**	**JS-N**	**C**
channels (CSP-style) —Phase 13	✓	✓	×	×	×	×	×
linear types / use-after-move —Phase 14	✓	×	×	×	×	×	×
owned / pub fields —Phase 15	✓	×	×	×	×	×	×
effects `!{fs,ai,net,mut}` —Phase 12	✓	×	×	×	×	×	×
structured concurrency `scope` —Phase 17	✓	✓	×	×	×	×	×
transient cast at any-boundary —Phase 16	✓	×	×	×	×	×	×
session types (runtime-checked typed protocols)	✓	✓	×	×	×	×	×
protocol traces (order-only)	✓	✓	×	×	×	×	×
AIOS service registry (alias-resolved send)	✓	✓	×	×	×	×	×

Networking, AI, infrastructure

Feature	Py-A	Py-I	OCaml	JS-O	JS-B	JS-N	C
remote actors (HTTP)	✓	✓	△no WS	△	×	△srv only	✓via OCaml
WebSocket	×	×	×	×	×	×	×
AI integration (`ai_call`)	✓stream/img	✓	✓	✓	✓mock	✓mock	✓
AI governance (budget/concurrent/fallback)	✓	✓	✓	✓	×	×	✓
HMAC-signed remote send	✓	✓	×	×	×	×	✓
persistent actor state	✓	✓	×	×	×	×	×
live dashboard (SSE)	✓	✓	△polling	△	×	×	✓
`/api/typecheck` JSON endpoint	×	×	✓	✓	×	✓	–
`/api/run` JSON endpoint	×	×	×	×	×	✓	–

Actor concurrency model

Every AIPL implementation runs each actor as a separate concurrent unit, but the kind of unit differs by 3–4 orders of magnitude in weight. This affects how many actors a program can usefully spawn, how messages interleave, and whether blocking I/O in one actor stalls the whole runtime.

Runtime / codegen target	Model	Per-actor unit	Source pointer
Python (annotated)	**1:1 OS thread**	`threading.Thread(daemon=True)`	`aipl_runtime.py:92`
Python (inferred)	**1:1 OS thread**	(inherits from python-aipl)	(re-uses interp)
OCaml	**1:1 OS thread**	`Thread.create`	`eval_thread.ml:~1168`
JS-OCaml (server)	**1:1 OS thread**	(= OCaml backend threads)	(= OCaml)
JS-Browser (browser-abcl)	**cooperative event loop**	`setTimeout`-driven mailbox drain	`runtime.js:~233`
JS-Node (node-aipl-server)	**cooperative event loop**	(= browser-abcl runtime)	re-export
C (default)	**1:1 OS thread**	`pthread_create`	`c_translator.ml:~721,790`
C + SDL2	**1:1 OS thread**	`pthread_create`	(same as default C)
C + Xinu	**1:1 OS process** (kernel-level)	Xinu `create()` + `ready()`	`c_translator.ml:~1077`
C → Python codegen	**1:1 OS thread**	`threading.Thread` (`o._spawn()`)	`c_translator.ml:~1847`
C → Pony codegen	**M:N lightweight** (runtime-scheduled)	Pony actor (work-stealing)	implicit in generated `actor`

Runtime / codegen target	Model	Per-actor unit	Source pointer
C → Erlang codegen	**M:N lightweight** (BEAM)	BEAM process (`spawn(fun()->...end)`)	`c_translator.ml:~2413`
C → Go codegen	**M:N lightweight** (Go runtime)	goroutine (`go c.run()`)	`c_translator.ml:~2728`

Three concurrency tiers

- **1:1 OS thread / process** (Python, OCaml, C, C-SDL2, Xinu, C→Python): each actor is a kernel-scheduled thread. Simple mental model; blocking I/O in one actor doesn't stall others. Scales to ∼100–1000 actors before kernel overhead dominates.
- **M:N lightweight** (Pony, Erlang, Go): actors are scheduled by the language runtime onto a small pool of OS threads. Scales to $\sim 10^5$–10^7 actors. Blocking syscalls in one actor may stall its scheduler unless the runtime intercepts them (Erlang's BEAM intercepts; Go's runtime intercepts via `netpoll`; Pony's runtime requires non-blocking idioms).
- **Cooperative event loop** (browser-abcl, node-aipl-server): single OS thread; mailboxes are drained turn-by-turn through `setTimeout(0)` re-entry into the event loop. No real parallelism, but message-level interleaving is preserved. A long synchronous computation in one actor *will* stall all others — by design, since this is the only concurrency model the browser DOM allows.

Cross-cutting implication

The AIPL programming model (`send` / `now` / `future` / `await`) is preserved across all three tiers — the choice of target picks a point on the cost/scale curve without changing the program text. The same `.abcl` file:

- runs ∼1000 actors fine on Python / OCaml / C (OS-thread tier);
- scales to millions of actors on Erlang / Go / Pony codegen;
- runs in a browser sandbox via JS-Browser at the cost of single-threaded execution.

C-version-specific codegen targets

Target	C
C + pthread standalone binary	✓
C + SDL2 GUI binary (1178-line runtime)	✓
Xinu embedded-OS target (`--xinu`)	✓
Python target (`--python`)	✓
Pony target (`--pony`) —`class`→`actor`, methods→`be`, two-step `_aipl_init`	✓
Erlang target (`--erlang`) —`class`→`spawn`+`receive` loop, fields→versioned loop args	✓
Go target (`--go`) —`class`→struct+goroutine, methods→typed message structs + type-switch dispatch	✓

Key observations

Python (annotated) is the most feature-complete runtime

Of the 30 surveyed features, Python (annotated) ticks roughly 28. It owns the full Phase 11–17 stack (channels, linear types, owned fields, effects, structured concurrency, transient casts), method injection, dynamic compile, persistent actor state, and full AI integration (including streaming and images).

Python (inferred) is Python's HM-typed twin

Runtime features match Python (annotated) exactly because the interpreter is shared. The trade is on the static side: the Phase 11+ annotation-driven checks are replaced with a Hindley-Milner inference pass. Inferred types cross-verified against OCaml: on shared samples (`Hello.abcl`, `counter.abcl`) the two runtimes produce identical type schemes.

OCaml is the canonical core

Implements roughly 15 of the surveyed features. Phase 11+ exist only as `.abcl` design-document samples, not implemented. Solid `become` / `select` / HM inference; remote is HTTP only (no HMAC, no WebSocket).

JS-OCaml ≈ OCaml

Thin HTTP client over the OCaml backend, so features inherit. The only differentiator is the `/api/typecheck` endpoint added in commit `5227f87`.

JS-Browser is the minimal implementation

Around 8 of the 30 features. Basic actor model plus our newly-added flow-sensitive type inference. No `become`, no channels, no remote. AI integration exists but is mock-only.

JS-Node (Node-server, newest)

Wraps the browser-abcl parser, type-checker, and interpreter inside a Node.js HTTP server. Exposes `/api/typecheck` (the same JSON shape as OCaml's endpoint) and `/api/run` (executes and returns stdout) —the only runtime with the latter. Runtime feature set mirrors browser-abcl; the value-add is having both the type-checker and the interpreter reachable over HTTP without bringing up either OCaml or a browser.

C version — surprisingly strong on infrastructure

About 17 of 30. `become` and `select` are not implemented (codegen emits `/* unsupported */`), but it inherits OCaml's HMAC, SSE, AI governance, and adds **three additional codegen targets** (SDL2, Xinu, Python). Fields, parameters, and locals are specialized to native C types.

Method injection — a Python-family exclusive

Both Python variants use mutable method dispatch tables that can be mutated at runtime via `add_method` / `remove_method`. Other runtimes substitute `become` (whole-class swap) as the closest equivalent. `JS-Browser` and `JS-Node` lack even `become` (pure actor execution).

Type inference cross-verification

For shared samples (`abclc/Hello.abcl`, `abclc/counter.abcl`), the three HM-based runtimes (**OCaml**, **Python-inferred**, **C**) produce identical inferred types:

```
Hello:   count : float
         init  : (int) -> unit
         greet : () -> unit
         inc   : () -> unit

Counter: count : float
         inc   : () -> unit
         dec   : (float) -> unit    <- monomorphized via call site
```

The flow-sensitive variants (**JS-Browser**, **JS-Node**) reach the same answers on shared samples within the limits of their algorithm —field types match, but method signature schemes are not produced the same way.

Phase 11+ implementation gap

The full Phase 11–17 stack is implemented only in Python (annotated). This represents AIPL's language-evolution frontier; Python (annotated) serves as the research runtime. Python (inferred) keeps the runtime side but trades the static checks for HM-style polymorphic inference — a different research axis (declarative typing vs. annotated capabilities).

Generated 2026-05-13. Includes `python-aipl-inferred` (commit `270f291`) and `node-aipl-server` (commit `adeb0b6`). For source pointers, run `grep` against the files listed in each runtime's source column.